WEEK 9: SALES AND CUSTOMER ANALYTICS SYSTEM

AIM

To analyze the sales data of the ArtHub database using SQL aggregate functions and generate reports to understand customer purchasing behaviour, artwork performance, sales quantity, and revenue trends.

INTRODUCTION

The ArtHub database is designed to manage artworks, customers, orders, payments, and sales information. Sales and customer analytics help to understand the purchasing behaviour of customers and the performance of different artworks.

SQL aggregate functions such as COUNT (), SUM (), AVG (), MIN (), and MAX () are used to perform calculations on multiple records and generate summarized results. These functions help identify total sales, average order value, highest and lowest order amounts, frequent customers, and best-selling artworks.

In this report, SQL operations are used to analyze ArtHub sales data and generate business analytics reports.

1. APPLY SQL AGGREGATE FUNCTIONS

Aggregate functions perform calculations on multiple records and return a single summarized value.

Function	Purpose	Example in ArtHub
COUNT()	Counts the number of records	Total number of orders
SUM()	Calculates the total value	Total sales amount
AVG()	Finds the average value	Average order amount
MIN()	Finds the minimum value	Lowest order amount
MAX()	Finds the maximum value	Highest order amount

These functions are used in the following sections to analyze sales and customer information.

2. GENERATE TOTAL SALES REPORTS

1. Total number of orders placed

The total number of orders placed by customers in ArtHub is calculated using the COUNT() aggregate function. It counts all the records available in the Orders table.

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS FROM Orders;

TOTAL_ORDERS
------------
           7
```

2. Total revenue generated from sales

The total revenue generated from all orders is calculated using the SUM() function. It adds the total amount of every order recorded in the Orders table.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM Orders;

TOTAL_REVENUE
-------------
        24500
```

3. Average order value

The average order value is calculated using the AVG() function. It finds the average of all order amounts recorded in the Orders table.

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM Orders;

AVERAGE_ORDER_VALUE
-------------------
               3500
```

4. Highest and lowest order amount

The MAX() function identifies the highest order amount, while the MIN() function identifies the lowest order amount recorded in ArtHub.

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_AMOUNT, MIN(TOTAL_AMOUNT) AS LOWEST_ORDER_AMOUNT FROM Orders;

HIGHEST_ORDER_AMOUNT LOWEST_ORDER_AMOUNT
-------------------- -------------------
                5200                1800
```

5. Total sales for a specific period

The total sales generated between 20 September 2026 and 24 September 2026 are calculated by filtering orders based on the specified date range. The SUM() function is used to calculate the total sales amount for this period.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES FROM Orders WHERE ORDER_DATE >= DATE '2026-09-20' AND ORDER_DATE < DATE '2026-09-25';

TOTAL_SALES
-----------
      24500
```

3. CUSTOMER PURCHASE ANALYSIS

1. Total number of orders placed by each customer

The total number of orders placed by each customer is calculated using the COUNT() function. The Customer and Orders tables are joined to display the order count for every customer. Customers who have not placed any orders are also included.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS TOTAL_ORDERS FROM Customer LEFT JOIN Orders ON Customer.Customer_ID = Or
ders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY Customer.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_ORDERS
----------- ----------------------------- ------------
          1 Dhoni                                    1
          2 Virat                                    1
          3 Ashwin                                   1
          4 SanjuSamson                              1
          5 Shreyas Iyer                             1
          6 Hardhik Pandya                           1
          7 Ruturaj Gaikwad                          1
          8 Rohit Sharma                             0
          9 Raina                                    0
         10 Jadeja                                   0

10 rows selected.
```

2. Total amount spent by each customer

The total amount spent by each customer is calculated using the SUM() function. It adds the order amounts for each customer. The NVL() function replaces NULL values with zero for customers who have not made any purchases.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, NVL(SUM(Orders.Total_Amount), 0) AS TOTAL_AMOUNT_SPENT FROM Customer LEFT JOIN Orders ON Customer.
Customer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY Customer.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_AMOUNT_SPENT
----------- ------------------------------ ------------------
          1 Dhoni                                        5000
          2 Virat                                        5200
          3 Ashwin                                       1800
          4 SanjuSamson                                  2200
          5 Shreyas Iyer                                 3500
          6 Hardhik Pandya                               4000
          7 Ruturaj Gaikwad                              2800
          8 Rohit Sharma                                    0
          9 Raina                                           0
         10 Jadeja                                          0

10 rows selected.
```

3. Average spending per customer

The average spending of each customer is calculated using the AVG() function. It finds the average order amount for every customer. Customers without orders are displayed with zero.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, NVL(AVG(Orders.Total_Amount), 0) AS AVERAGE_SPENDING FROM Customer LEFT JOIN Orders ON Customer.Cu
stomer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY Customer.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME                  AVERAGE_SPENDING
----------- ------------------------------ ----------------
          1 Dhoni                                      5000
          2 Virat                                      5200
          3 Ashwin                                     1800
          4 SanjuSamson                                2200
          5 Shreyas Iyer                               3500
          6 Hardhik Pandya                             4000
          7 Ruturaj Gaikwad                            2800
          8 Rohit Sharma                                  0
          9 Raina                                         0
         10 Jadeja                                        0

10 rows selected.
```

4. Customers with maximum purchase amount

This analysis identifies the customer or customers who have spent the highest total amount on ArtHub orders. The total spending of each customer is calculated and compared to identify the maximum purchase amount.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, SUM(Orders.Total_Amount) AS TOTAL_PURCHASE_AMOUNT FROM Customer INNER JOIN Orders ON Customer.Cust
omer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING SUM(Orders.Total_Amount) = (SELECT MAX(TOTAL_SPENDING) FROM (SELEC
T SUM(Total_Amount) AS TOTAL_SPENDING FROM Orders GROUP BY Customer_ID));

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_PURCHASE_AMOUNT
----------- ------------------------------ --------------------
          2 Virat                                          5200
```

5. Customers with fewer purchases

This analysis identifies customers who have placed one or fewer orders. The number of orders placed by each customer is counted to identify customers with fewer purchases.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS TOTAL_ORDERS FROM Customer LEFT JOIN Orders ON Customer.Customer_ID = Or
ders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING COUNT(Orders.Order_ID) <= 1 ORDER BY TOTAL_ORDERS;

CUSTOMER_ID CUSTOMER_NAME                  TOTAL_ORDERS
----------- ------------------------------ ------------
          8 Rohit Sharma                              0
          9 Raina                                     0
         10 Jadeja                                    0
          5 Shreyas Iyer                              1
          2 Virat                                     1
          7 Ruturaj Gaikwad                           1
          3 Ashwin                                    1
          1 Dhoni                                     1
          4 SanjuSamson                               1
          6 Hardhik Pandya                            1

10 rows selected.
```

4. FIND TOP CUSTOMERS BASED ON PURCHASE AMOUNT

1. Top 5 customers based on total spending

This analysis displays the top five customers based on their total spending. The total purchase amount of each customer is calculated and arranged in descending order to identify the highest-spending customers.

```
SQL> SELECT * FROM (SELECT Customer.Customer_ID, Customer.Customer_Name, SUM(Orders.Total_Amount) AS TOTAL_SPENDING FROM Customer INNER JOIN Orders ON Custo
mer.Customer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY TOTAL_SPENDING DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_SPENDING
----------- ----------------------------- --------------
          2 Virat                                   5200
          1 Dhoni                                   5000
          6 Hardhik Pandya                          4000
          5 Shreyas Iyer                            3500
          7 Ruturaj Gaikwad                         2800
```

2. Customers with maximum number of orders

This analysis identifies the customer or customers who have placed the maximum number of orders in ArtHub. The total number of orders placed by each customer is calculated and compared.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS TOTAL_ORDERS FROM Customer INNER JOIN Orders ON Customer.Customer_ID = O
rders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING COUNT(Orders.Order_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS OR
DER_COUNT FROM Orders GROUP BY Customer_ID));

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_ORDERS
----------- ----------------------------- ------------
          4 SanjuSamson                              1
          1 Dhoni                                    1
          6 Hardhik Pandya                           1
          3 Ashwin                                   1
          5 Shreyas Iyer                             1
          2 Virat                                    1
          7 Ruturaj Gaikwad                          1

7 rows selected.
```

3. Frequent customers

This analysis identifies frequent customers who have placed more than one order in ArtHub. The number of orders placed by each customer is calculated to identify customers with repeated purchases.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS TOTAL_ORDERS FROM Customer INNER JOIN Orders ON Customer.Customer_ID = O
rders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING COUNT(Orders.Order_ID) > 1 ORDER BY TOTAL_ORDERS DESC;

no rows selected
```

4. High-value customers

This analysis identifies high-value customers whose total purchase amount is ₹4,000 or more. The total spending of each customer is calculated to identify customers who contribute more to ArtHub sales.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, SUM(Orders.Total_Amount) AS TOTAL_SPENDING FROM Customer INNER JOIN Orders ON Customer.Customer_ID
 = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name HAVING SUM(Orders.Total_Amount) >= 4000 ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME                 TOTAL_SPENDING
----------- ----------------------------- --------------
          2 Virat                                   5200
          1 Dhoni                                   5000
          6 Hardhik Pandya                          4000
```

5. IDENTIFY BEST-SELLING PRODUCTS

In ArtHub, the products are represented by artworks stored in the ArtProduct table. The OrderDetails table contains the quantity and price information required for sales analysis.

1. Products with maximum sales quantity

This analysis identifies the artwork or artworks with the highest total quantity sold. The total quantity sold for each artwork is calculated to identify the best-selling products.

```
SQL> SELECT ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, SUM(OrderDetails.Quantity) AS TOTAL_QUANTITY_SOLD FROM ArtProduct INNER JOIN OrderDetails ON Art
Product.Artwork_ID = OrderDetails.Artwork_ID GROUP BY ArtProduct.Artwork_ID, ArtProduct.Artwork_Name HAVING SUM(OrderDetails.Quantity) = (SELECT MAX(TOTAL_Q
UANTITY) FROM (SELECT SUM(Quantity) AS TOTAL_QUANTITY FROM OrderDetails GROUP BY Artwork_ID));

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
104
Charcoal Dreams
                  1

102
Colors of Nature
                  1

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

106
Future City
                  1

101
Sunset Over Hills

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1
```

2. Products generating highest revenue

This analysis calculates the revenue generated by each artwork by multiplying the quantity sold by its price. The total revenue is calculated and arranged in descending order to identify artworks generating the highest revenue.

```
SQL> SELECT ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, SUM(OrderDetails.Quantity * OrderDetails.Price) AS TOTAL_REVENUE FROM ArtProduct INNER JOIN Orde
rDetails ON ArtProduct.Artwork_ID = OrderDetails.Artwork_ID GROUP BY ArtProduct.Artwork_ID, ArtProduct.Artwork_Name ORDER BY TOTAL_REVENUE DESC;

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
102
Colors of Nature
         5200

101
Sunset Over Hills
         5000

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------

106
Future City
         4000

105
Digital Galaxy

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_REVENUE
-------------
         3500
```

3. Least-selling products

This analysis identifies artworks with the lowest sales quantity. Artworks without matching sales records are also considered. This helps identify products that have not performed well in the market.

```
SQL> SELECT ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, NVL(SUM(OrderDetails.Quantity), 0) AS TOTAL_QUANTITY_SOLD FROM ArtProduct LEFT JOIN OrderDetails
 ON ArtProduct.Artwork_ID = OrderDetails.Artwork_ID GROUP BY ArtProduct.Artwork_ID, ArtProduct.Artwork_Name HAVING NVL(SUM(OrderDetails.Quantity), 0) = (SEL
ECT MIN(TOTAL_QUANTITY) FROM (SELECT NVL(SUM(Quantity), 0) AS TOTAL_QUANTITY FROM OrderDetails GROUP BY Artwork_ID)) ORDER BY ArtProduct.Artwork_ID;

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
101
Sunset Over Hills
                  1

102
Colors of Nature
                  1

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

103
Pencil Portrait
                  1

104
Charcoal Dreams

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1
```

4. Products requiring promotion

This analysis identifies artworks with zero or one item sold. These artworks may require promotional offers or marketing activities to improve their sales performance.

```
SQL> SELECT ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, NVL(SUM(OrderDetails.Quantity), 0) AS TOTAL_QUANTITY_SOLD FROM ArtProduct LEFT JOIN OrderDetails
 ON ArtProduct.Artwork_ID = OrderDetails.Artwork_ID GROUP BY ArtProduct.Artwork_ID, ArtProduct.Artwork_Name HAVING NVL(SUM(OrderDetails.Quantity), 0) <= 1 O
RDER BY TOTAL_QUANTITY_SOLD;

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
109
Abstract Dreams
                  0

107
Morning Light
                  0

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

108
The Thinker Sculpture
                  0

110
Handmade Craft

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  0

105
```

6. PERFORM CATEGORY-WISE SALES ANALYSIS

Category-wise sales analysis helps to understand the sales performance of different artwork categories. The CATEGORY_ID column in the ArtProduct table is used to group artworks by category.

1. Total sales generated by each category

This analysis calculates the total sales revenue generated by each artwork category. The quantity sold is multiplied by the price, and the revenue is added to calculate the total sales for each category.

```
SQL> SELECT ArtProduct.CATEGORY_ID, SUM(OrderDetails.QUANTITY * OrderDetails.PRICE) AS TOTAL_SALES FROM ArtProduct INNER JOIN OrderDetails ON ArtProduct.ART
WORK_ID = OrderDetails.ARTWORK_ID GROUP BY ArtProduct.CATEGORY_ID ORDER BY TOTAL_SALES DESC;

CATEGORY_I TOTAL_SALES
---------- -----------
1               10200
3                7500
2                4000
```

2. Number of products sold in each category

This analysis calculates the total number of artwork units sold in each category. The quantities of artworks sold under each category are added to obtain the total products sold.

```
SQL> SELECT ArtProduct.CATEGORY_ID, SUM(OrderDetails.QUANTITY) AS TOTAL_PRODUCTS_SOLD FROM ArtProduct INNER JOIN OrderDetails ON ArtProduct.ARTWORK_ID = Ord
erDetails.ARTWORK_ID GROUP BY ArtProduct.CATEGORY_ID ORDER BY TOTAL_PRODUCTS_SOLD DESC;

CATEGORY_I TOTAL_PRODUCTS_SOLD
---------- -------------------
1                            2
2                            2
3                            2
```

3. Highest revenue-generating category

This analysis identifies the category that generates the highest sales revenue. The total revenue of each category is calculated and compared to identify the highest revenue-generating category.

```
SQL> SELECT CATEGORY_ID, TOTAL_REVENUE FROM (SELECT ArtProduct.CATEGORY_ID, SUM(OrderDetails.QUANTITY * OrderDetails.PRICE) AS TOTAL_REVENUE FROM ArtProduct
 INNER JOIN OrderDetails ON ArtProduct.ARTWORK_ID = OrderDetails.ARTWORK_ID GROUP BY ArtProduct.CATEGORY_ID ORDER BY TOTAL_REVENUE DESC) WHERE ROWNUM = 1;

CATEGORY_I TOTAL_REVENUE
---------- -------------
1                  10200
```

4. Average sales per category

This analysis calculates the average sales value for each artwork category. The average sales amount is calculated to understand the sales performance of different categories.

```
SQL> SELECT ArtProduct.CATEGORY_ID, AVG(OrderDetails.QUANTITY * OrderDetails.PRICE) AS AVERAGE_SALES FROM ArtProduct INNER JOIN OrderDetails ON ArtProduct.A
RTWORK_ID = OrderDetails.ARTWORK_ID GROUP BY ArtProduct.CATEGORY_ID ORDER BY AVERAGE_SALES DESC;

CATEGORY_I AVERAGE_SALES
---------- -------------
1                  5100
3                  3750
2                  2000
```

7. GENERATE BUSINESS ANALYTICS REPORTS

Report 1: Sales Performance Report

This report includes:

- Total sales
- Number of orders
- Average order value
- Highest order value

The Orders table is used to generate the overall sales performance report. The total sales, number of orders, average order value, and highest order value are calculated using aggregate functions. This report helps evaluate the overall sales performance of ArtHub.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES, COUNT(ORDER_ID) AS NUMBER_OF_ORDERS, AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE, MAX(TOTAL_AMOUNT) AS HIGHEST_OR
DER_VALUE FROM Orders;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
      24500                7                3500                5200
```

Report 2: Customer Analytics Report

This report includes:

- Customer name
- Number of orders

- Total spending
- Purchase frequency

The Customer and Orders tables are joined to generate the customer analytics report. The number of orders and total spending of each customer are calculated. Customers are classified according to their purchase frequency. Customers who have not placed any orders are also included in the report.

```
SQL> SELECT Customer.Customer_ID, Customer.Customer_Name, COUNT(Orders.Order_ID) AS NUMBER_OF_ORDERS, NVL(SUM(Orders.Total_Amount), 0) AS TOTAL_SPENDING, CA
SE WHEN COUNT(Orders.Order_ID) = 0 THEN 'NO PURCHASES' WHEN COUNT(Orders.Order_ID) = 1 THEN 'OCCASIONAL' ELSE 'FREQUENT' END AS PURCHASE_FREQUENCY FROM Cust
omer LEFT JOIN Orders ON Customer.Customer_ID = Orders.Customer_ID GROUP BY Customer.Customer_ID, Customer.Customer_Name ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_SPENDING
----------- ------------------------------ ---------------- --------------
PURCHASE_FRE
------------
          2 Virat                                         1           5200
OCCASIONAL

          1 Dhoni                                         1           5000
OCCASIONAL

          6 Hardhik Pandya                                1           4000
OCCASIONAL


CUSTOMER_ID CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_SPENDING
----------- ------------------------------ ---------------- --------------
PURCHASE_FRE
------------
          5 Shreyas Iyer                                  1           3500
OCCASIONAL

          7 Ruturaj Gaikwad                               1           2800
OCCASIONAL

          4 SanjuSamson                                   1           2200
OCCASIONAL


CUSTOMER_ID CUSTOMER_NAME                  NUMBER_OF_ORDERS TOTAL_SPENDING
----------- ------------------------------ ---------------- --------------
PURCHASE_FRE
------------
          3 Ashwin                                        1           1800
OCCASIONAL

         10 Jadeja                                        0              0
```

Report 3: Product Performance Report

This report includes:

- Product name
- Quantity sold
- Revenue generated

The ArtProduct and OrderDetails tables are joined to calculate the quantity sold and total revenue for each artwork. The SUM() function calculates the total quantity sold and revenue. The results are grouped by artwork and arranged in descending order of revenue.

```
SQL> SELECT ArtProduct.Artwork_ID, ArtProduct.Artwork_Name, NVL(SUM(OrderDetails.Quantity), 0) AS QUANTITY_SOLD, NVL(SUM(OrderDetails.Quantity * OrderDetail
s.Price), 0) AS REVENUE_GENERATED FROM ArtProduct LEFT JOIN OrderDetails ON ArtProduct.Artwork_ID = OrderDetails.Artwork_ID GROUP BY ArtProduct.Artwork_ID,
ArtProduct.Artwork_Name ORDER BY REVENUE_GENERATED DESC;

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
102
Colors of Nature
            1              5200

101
Sunset Over Hills
            1              5000

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------

106
Future City
            1              4000

105
Digital Galaxy

ARTWORK_ID
----------
ARTWORK_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
            1              3500
```

This report helps identify artworks with higher sales and revenue.

Report 4: Category Analysis Report

This report includes:

- Category ID
- Total products sold
- Total revenue

The ArtProduct and OrderDetails tables are joined to generate a category-wise analysis report. The total quantity of artworks sold and the total revenue generated by each category are calculated. The results are grouped according to CATEGORY_ID.

```
SQL> SELECT ArtProduct.CATEGORY_ID, SUM(OrderDetails.QUANTITY) AS TOTAL_PRODUCTS_SOLD, SUM(OrderDetails.QUANTITY * OrderDetails.PRICE) AS TOTAL_REVENUE FROM
  ArtProduct INNER JOIN OrderDetails ON ArtProduct.ARTWORK_ID = OrderDetails.ARTWORK_ID GROUP BY ArtProduct.CATEGORY_ID ORDER BY TOTAL_REVENUE DESC;

CATEGORY_I TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
---------- ------------------- -------------
1                           2         10200
3                           2          7500
2                           2          4000
```

This report helps identify categories that contribute more to ArtHub sales.

CONCLUSION

Thus, the Week 9 Sales and Customer Analytics System was successfully implemented for the ArtHub database using SQL aggregate functions, joins, subqueries, and grouping operations.

The analysis was used to calculate total sales, average order value, highest and lowest order amounts, customer spending, purchase frequency, and artwork performance. Category-wise sales analysis and business analytics reports were also generated.

These reports help understand customer purchasing behaviour, identify best-selling artworks, evaluate sales performance, and support better business decisions in the ArtHub system.